首届 openJiuwen 智能体创新大赛

openJiuwen

软件作品技术说明书

作品名称	AcceptAnything
项目定位	工程项目全链路测试验收工具
参赛方向	Coding 方向
文档名称	工程设计与实现
文档版本	1.1
编制日期	2026 年 11 月 3 日

内容说明

本文说明仓库接入、验收方案生成、隔离执行、缺陷复核与质量门禁的实现。各章配有系统结构图、程序流程图、实现位置和实仓记录，供评审核查。

目　录

第一章　需求分析.....3
1.1　项目背景.....3
1.2　用户场景与输出.....3
1.3　与现有工程工具的关系.....3
第二章　概要设计.....4
2.1　系统组成与部署边界.....4
2.2　计划确认与运行创建时序.....5
第三章　详细设计.....6
3.1　仓库识别与计划依赖图.....6
3.2　独立依据与模型用例编译.....6
3.3　环境基线与多服务编排.....7
3.4　业务执行协议与观察值.....8
3.5　证据身份与归档结构.....9
3.6　扫描归一化、去重与复核.....9
3.7　结果契约与质量门禁.....10
3.8　任务持久化与中断恢复.....11
3.9　历史索引与读路径隔离.....12
第四章　工作台与业务实例.....13
4.1　运行上下文与分项导航.....13
4.2　Flask 教程删除流程的证据链.....14
第五章　测试报告.....14
5.1　实仓业务验收与结果归因.....14
5.2　反例回归与读取性能.....15
第六章　部署与使用.....16
6.1　单机运行环境.....16
6.2　主要接口与操作顺序.....17
第七章　项目总结.....17
7.1　工程贡献.....17
7.2　支持边界与后续验证.....17
附录　实现与资料索引.....18
A　代码与工程图定位.....18
B　验证材料与关联文档.....18

第一章　需求分析

1.1　项目背景

交付一个新版本时，代码通常以分支或提交提供，需求写在文档里，测试和安全扫描又各有报告。接收方需要先核对它们是否针对同一版本，再确认报告中的测试是否检验了约定的业务行为。缺少这种对应关系时，即使所有作业都结束，也无法回答某项需求是否达到交付条件。

AI 辅助开发进一步放大了预期来源的问题。模型读到某函数当前返回空列表，很容易把空列表写成预期值。这样的断言能重现实现，却不能证明实现符合需求。平台因此将独立需求作为预期的依据，将代码中的签名、路由、启动声明等只用来连接和运行被测对象。

1.2　用户场景与输出

表 1.2　使用者与交付材料

使用者	实际任务	所需材料
交付负责人	核对合同或需求中指定的业务流程，确认可否接收版本	仓库标识、冻结范围、逐项覆盖、门禁理由
测试负责人	审阅预期依据，定位用例、环境或平台问题	引用原文、用例步骤、执行尝试及前置状态
开发者	复现问题并比较修复后的结果	失败位置、请求与响应、日志、原件、历史运行

1.3　与现有工程工具的关系

构建工具负责产出构建结果，测试框架执行断言，安全工具报告候选风险。AcceptAnything 复用这些工具，把各自的结果放回共同的版本和需求范围。以登录流程为例，构建成功、路由可用、浏览器登录后识别到当前用户，分别是不同层次的观察。报告分别保留这些结论。GitLab 的 JUnit 报告不会直接改变作业状态；Allure 区分断言失败与测试损坏；Playwright Trace Viewer 可以回看浏览器操作。本项目进一步将这些工程观察关联到独立需求，再评估整个冻结范围能否放行。官方文档见附录。

第二章　概要设计

2.1　系统组成与部署边界

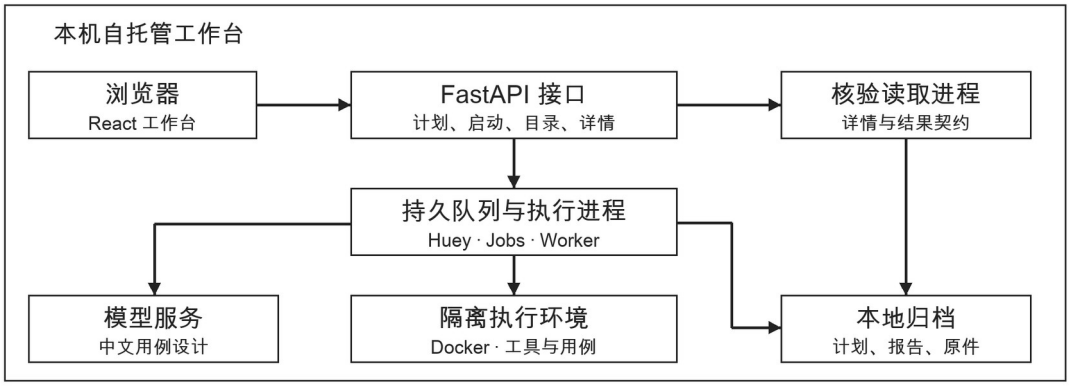


图 2.1　单机部署结构

浏览器工作台由 React 组成，通过 FastAPI 提交方案、启动任务、读取目录及详情。API 创建运行后将作业交给持久队列。Worker 取得执行权后调度适配器，测试过程与模型调用在该执行路径中完成。模型只在方案设计环节参与，对外报告的门禁结论由平台规则生成。

表 2.1　系统组件及实现职责

组件	实现与责任
接入与工作台	api.py 负责请求分派；Workbench、ConsoleFlow、Catalogue 展示统一视图、检查进度和历史目录。
执行控制	Jobs 保存作业记录和事件；SqliteHuey 投递任务。不同执行类型由对应 Store 读取和执行。
执行与核验	通用运行时准备环境并调用工具；集成存储保存任务、报告及原件。详情核验可交给独立读取进程。

本图以新建通用验收为主。既有 native 运行及显式配置仍有独立实现，统一详情负责投影兼容历史记录。迭代通用协议无需改写已归档运行。

实现位置　api.py / full_plan；jobs.py / Jobs；read_service.py / ProofReaders

2.2 计划确认与运行创建时序

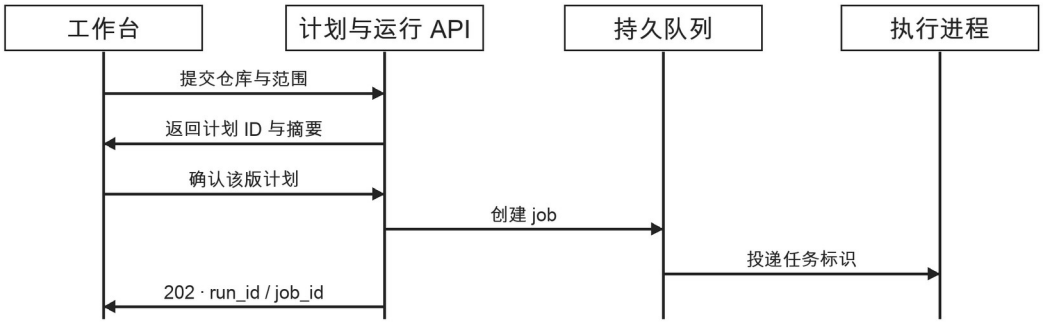


图 2.2　新建通用运行的消息时序

第一步提交仓库位置及版本选择。API 根据输入选择远程取源、本地路径、显式配置或历史设计复用。常规通用入口调用 prepare，生成执行单元、文档分批、检查项与冻结摘要。与引擎通信的是结构化计划。

第二步提交 plan_id 和 digest。create 会重新校验源清单、实现身份、工具链锁摘要；远端来源还要复验取源记录。通过后保存 universal-plan.json，将本次驱动文件同时归档，再入队。若准备计划后文件改变或执行器升级，需重新识别，避免用旧方案去测新内容。

表 2.2　计划与运行标识

标识	用途	失配后果
plan_id 与 digest	定位并核对用户确认的方案	拒绝创建或要求重新规划
run_id	关联一个固定对象的全部执行及原件	其他运行的原件不能直接作为本轮凭据
job_id 与 attempt_id	追查调度状态、单次领取与恢复事件	重复投递不能自行开启第二次执行

HTTP 202 表示任务已被接受。是否开始、是否结束、得到的业务结论，分别要看作业、运行及结果契约。

实现位置　api.py / full_start；service.py / create；jobs.py / enqueue

第三章　详细设计

3.1　仓库识别与计划依赖图

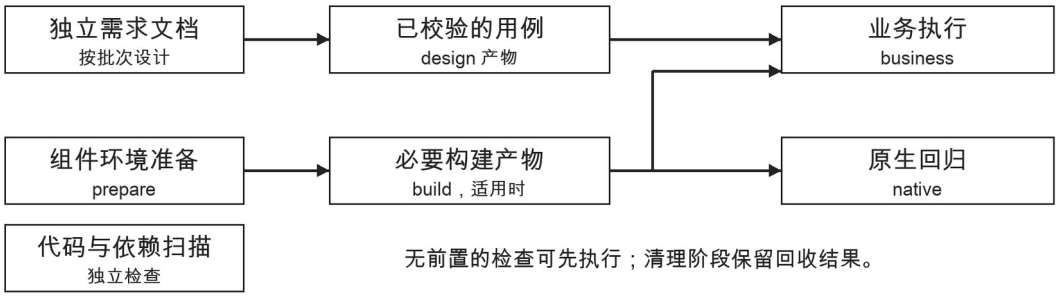


图 3.1　单个执行组件的典型依赖

远程分支先经 ls-remote 解析到提交，再按 SHA 浅取并核对 FETCH_HEAD。检出前检查树条目；发现符号链接或子模块时拒绝当前远端接入。LFS 指针缺少实际对象也不进入验收。源码快照摘要还要覆盖实际纳入的文件，Git 提交号与本次检查范围分别保存。

发现模块根据项目清单识别 Python、Node、Go、Rust 等组件，建立组件归属与本地依赖。Python 的公开导出索引递归解析别名；缓存键包括模块和访问路径集合，使循环导出能终止且不同根目录不会共享旧结果。接口索引帮助定位调用入口，不能拿来推导业务预期。

生成 DAG 时，prepare 先依赖本地组件的准备任务。需要构建产物的分支再依赖 build；独立的语法检查失败不应抑制无关用例。模型设计按资料批次入图，扫描作为独立任务，清理排到最后。执行时读取当前节点 needs 检查实际回执，图上的一条边不代表节点已经执行。

实现位置　remote.py / checkout；discovery.py / python_exports；service.py / prepare

3.2　独立依据与模型用例编译

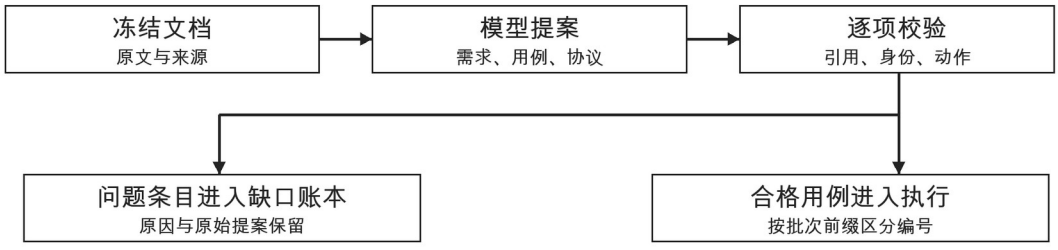


图 3.2　模型提案的接纳与拒绝

文档发现建立目录，解析项目内链接、目录树及章节片段，保留来源。单次计划给材料分配 D1、D2 等短编号；编号须与相应文档内容及摘要一起解释。送给模型的接口元数据只包含与本批资料相关的执行信息，不把原有测试的答案放入预期。

validate 先读取 Proposal，再逐项检查。需求引用必须能按冻结协议定位；当前协议检查唯一性。中文标题、编号唯一性、组件能力、公开调用入口和服务依赖均有独立约束。无效用例保存原因，其余有效条目可继续处理。批间编号加前缀，以免模型在两个批次里都使用相同的局部编号。

表 3.2　模型输出异常的处理

情形	处理方式
模型给出不存在或无法唯一定位的引文	记录引用问题，不将该条文案用作验收依据。
完整响应仅有 JSON 语法错误	满足身份与回执条件时至多重新生成一次，两次材料分别保留。
请求已发送但上游终态未知	保留未知；不把超时当成失败回执，也不自动重复付费调用。
使用已归档的模型原件复验	标记 frozen-response-replay，保存原始设计来源；执行与清理仍产生新记录。

实现位置　documents.py / enrich；generation.py / validate；service.py / design_draft

3.3　环境基线与多服务编排

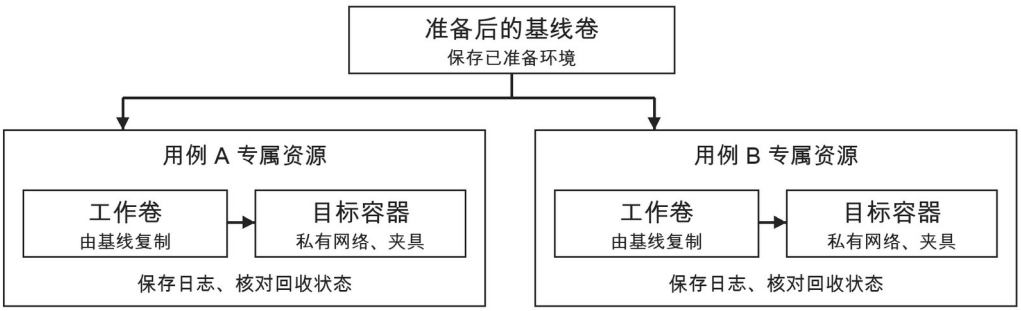


图 3.3　用例隔离边界

依赖安装和准备完成后，平台为组件生成基线卷。copy_volume 核对源卷所有权，要求目标卷尚不存在，再以只读挂载复制源数据。在独立用例执行路径中，allocate 使用用例 ID 摘要派生资源名，创建专属卷和网络，以只读根文件系统运行目标。被测程序在可写工作卷中操作，上一条用例创建的账户或数据库文件不会成为下一条的隐含前提。

服务发现仅接受有来源的声明。当前目录能识别 Redis、PostgreSQL 和 MongoDB 的本地端点，但自动就绪分支对端点类型的支持更窄：默认端口、无凭据的本地 Redis 才满足当前自动条件。Celery 启动命令还需通过参数检查并确认模块文件存在。数据库目录项不能直接算作已支持的部署拓扑。

运行时按依赖顺序启动服务，保存探针参数、退出码与输出。Redis 要返回 PONG；Celery 要从本次 worker 名称得到 pong。进入用例前检查 requires 是否包含于就绪集合。退出时逆序清理服务，检查容器、网络和卷是否仍存在，并写入 environment.json。

实现位置　orchestration.py / discover；orchestration.py / start；case_environment.py / allocate

3.4　业务执行协议与观察值

平台用有限动作描述模型用例。每个动作给出输入、前置状态和读回方式，执行器返回实际观察。工程人员据此可以区分调用未发生、调用发生但观察字段写错，以及结果确实与预期不同。

表 3.4　业务协议及观察范围

协议	输入与执行方式	判定边界
module / CLI	调用发现的公开函数或已声明的一次性命令	CLI 返回 exit_code、stdout、stderr。对输出文本的断言须选择 stdout。
scenario	通过 call、get、item、enter、exit 等步骤建立状态；用前序引用传递对象	区分属性和按键取值。只得到对象句柄，不说明业务状态符合预期。
HTTP / browser	在可启动服务上请求接口或操作页面，保存响应、定位信息及断言	进程内测试客户端通过，只能证明该入口；浏览器覆盖需真实浏览器执行。
journey	在独立环境内准备应用、执行连续动作并清理；可包含请求与轮询	对数量、可见性、状态等按类型校验；不能把刚捕获的响应值直接插为预期。

3.4.1　CLI 输出的反例

设命令退出码为 0，标准输出为一段 JSON。若用例把整份回执与期望字符串直接比较，比较对象就错了。协议要求明确选择 /stdout；若要验证进程是否正常结束，选择 /exit_code 并传入整数。前一项读取失败，不能替代后一项的判断。

3.4.2　设计代价

限制动作种类，可以在运行前发现错误的字段、未声明的服务和不安全的入口，但也会出现有充分需求依据、当前平台却无法表达的动作。这类情况应留作平台能力缺口；扩大协议需要补上对应反例和归档兼容校验。

实现位置　scenarios.py / validate_cli_checks；scenarios.py / evaluate；journeys.py / Journey

3.5 证据身份与归档结构

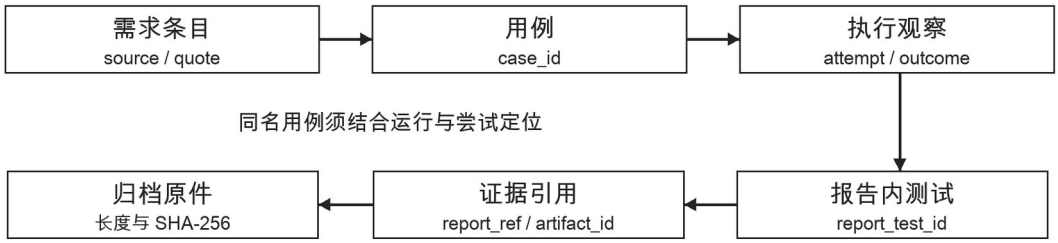


图 3.5　验收需求、用例、执行与原件的关联关系

结果契约中的 requirement 保存出处、引文及 case_ids；observation 保存执行任务、尝试、报告内测试 ID、需求引用和原件引用。两个运行可以使用同一用例名，客户端必须结合 run_id 定位记录。统一原件入口只在所选运行或与之绑定的子运行中查找。

表 3.5　归档对象及校验方式

归档对象	保存内容	读取校验
运行信封 run.json	运行标识、任务状态及摘要；较大的计划、报告可外置	信封摘要和运行 ID 一致。
payloads 目录	按内容 SHA-256 命名的 JSON 分片；相同内容可复用文件	检查摘要格式、符号链接和文件内容，再解析 JSON。
artifacts 与原件清单	原始报告、模型材料、日志、截图和 trace；记录 ID、名称、字节数、摘要	在返回内容前核对所属记录、大小和摘要。

写入 JSON 时先写临时文件、逐块计算摘要，再原子替换。重新核验发现损坏时保留异常，不能悄悄改用另一轮的同名报告。摘要用于本地一致性检查；若攻击者能同时重写原件和所有摘要，还需要外部签名或可信存储来证明来源。

实现位置　storage.py / compact；storage.py / expand；api.py / unified_artifact

3.6 扫描归一化、去重与复核

适配层分别解析 JUnit、SARIF、覆盖率、软件物料清单及专项报告。JUnit 的测试状态、SARIF 的规则命中和 SBOM 的组件清单有不同语义。统一格式保留工具、原始报告引用与关键字段，后续才能选择合适的复核方式。

3.6.1 指纹构造

adjudication.observations 对扫描命中与适用的失败测试构造稳定键。参与比较的字段包括类型、工具、环境、规则或测试组、规范化路径、行号、组件、名称、描述和严重度。编码后计算摘要。相同指纹归为同组，组内保留原 occurrence、task_id 和 report_ref；不同工具仅命中同一行或同一 CWE 时，不据此自动认定是同一问题。

表 3.6　扫描观察与复核计数

计数	含义
raw / unique / duplicates	原始观察数、唯一组数、重复观察数；重复数为前两者之差。
needs_review / confirmed	待核实影响与已有证据确认的组；页面分别展示。
false_positive / test_issue	规则误报与测试本身的问题；复核状态独立于 P0–P3 处置优先级。

3.6.2　测试断言反例

Bandit 的 B101 规则在测试代码中命中 assert。平台先核对冻结源码摘要，解析 pytest 路径和打包范围，亦可在测试文件中以 AST 核对测试函数里的断言行。满足规则条件才归入误报；普通字符串包含 assert 或文件名近似都不足以豁免。缺少可核验源码时保持待复核。人工意见绑定快照和指纹，后续更正以追加记录表达。

实现位置　formats.py / parse；adjudication.py / observations；adjudication.py / analyze

3.7　结果契约与质量门禁

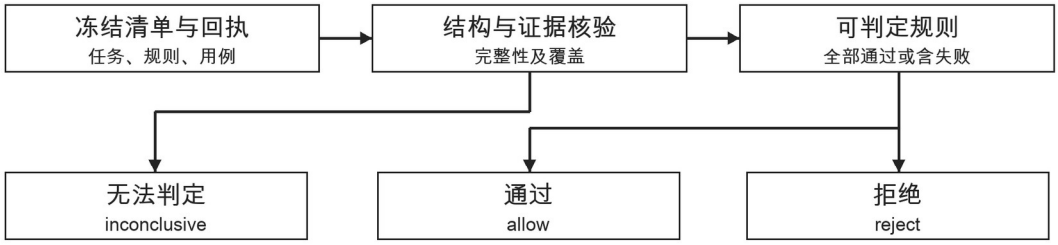


图 3.7　质量门禁判定流程

acceptance-result-v1 分别保存 execution、coverage、requirements、observations、findings、gate 和 provenance。gate_blockers 检查实际任务与计划任务的集合、规则身份、必需报告和报告引用。每条必验用例应恰有一份有效结果；跳过或异常须在报告中解释，不能为该需求补足覆盖。

表 3.7　门禁判定条件

输入情况	有效判定	示例
材料完整，适用规则均通过	allow	本次选定的范围、规则及证据均可核对。
材料完整，存在明确失败规则	reject	冻结规则要求失败数为零，完整回执中有失败断言。
必验未覆盖、缺原件或结构阻断	inconclusive	模型必验用例未执行；或任务已退出但报告遗失。

结构校验和 OPA 策略负责不同层次。前者检查能否下结论，后者根据规则结果决定放行或拒绝。旧记录保留 recorded_decision；重新读取时，若原件校验失效，当前 decision 不沿用历史通过。对同一需求，断言失败可构成有效的执行观察，但不会由此自动确认为产品缺陷。

3.7.1 策略实现节选

```
default decision := "inconclusive"
decision := "allow" if { ready; not failed }
decision := "reject" if { ready; failed }
```

摘自 acceptance/integrations/gates.rego。ready 汇集合约、完整性、任务报告及可判定规则等前提，failed 表示至少一条规则失败。

实现位置　results.py / gate_blockers；results.py / project；gates.py / opa_decide

3.8 任务持久化与中断恢复

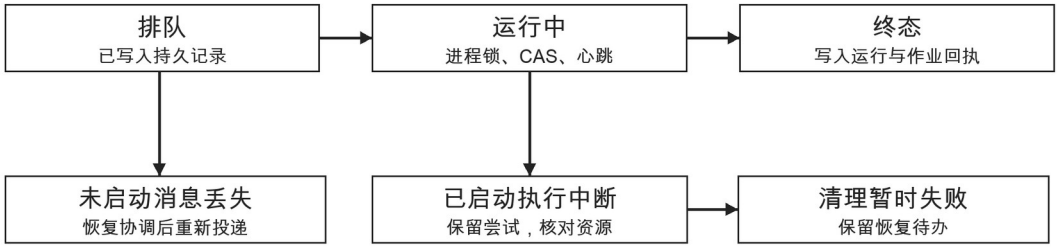


图 3.8　任务领取与恢复路径

API 提交后，作业身份写入持久记录。执行进程先取得 Store 的执行租约，再以条件更新将 queued 变为 running，同时写 attempt_id 和 lease_owner。已被领取的作业再次收到消息，不会重复进入执行。持锁期间后台心跳每约两秒更新一次，时间过期本身不能授权另一个执行进程抢占。

3.8.1 三个故障窗口

表 3.8　作业恢复分支

中断位置	恢复时的判据	处理
记录已入库、消息尚未送达	作业与运行均未开始	重新投递未开始任务，保留原标识。
工作进程失联，原运行已经启动	运行记录及进程锁显示旧尝试无法继续	先检查并清理有该运行所有权标签的资源；保留中断原因，重新执行需另建尝试。
运行终态已写、作业终态未写	磁盘已有终态证据	协调作业记录与已保存状态，避免再次执行业务。

取消同样受执行器能力约束。排队任务可以在开始前取消；已运行的旧 native 配方不支持协作式中止。服务接口明确拒绝这一操作，避免界面先显示取消而后台仍在运行。

实现位置　jobs.py / execute；jobs.py / recover；jobs.py / cancel

3.9　历史索引与读路径隔离

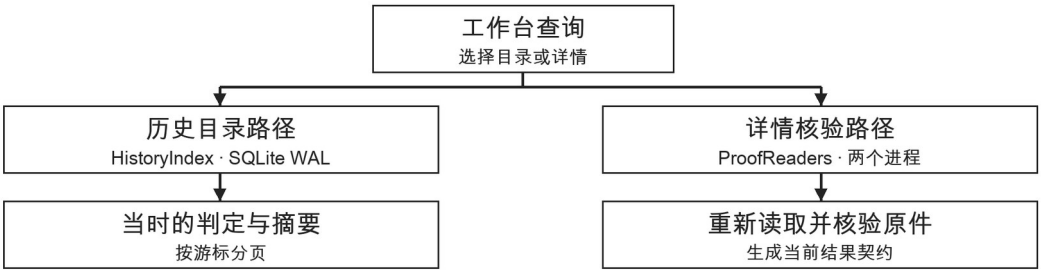


图 3.9　目录与详情读取路径

若目录页为展示几十行记录而逐份展开扫描报告、重新计算文件摘要，历史条目越多，列表响应越容易受单份大报告拖累。RecordIndex 采用 SQLite WAL 保存可检索的摘要及父子关系。后台同步按变化标记刷新；目录返回“当时判定”。历史索引可以重建，原始运行文件仍是核验依据。

目录游标包含排序锚点，并绑定查询条件。创建时间相同时再按唯一 ID 比较，翻页不依赖在变化的数据集中计算固定偏移。每页条数限制控制单次响应规模；该限制不等于运行记录总数上限。历史缺少聚合数时保留未知。

ProofReaders 使用 spawn 启动最多两个读取进程，主详情和结果契约的解析、散列计算与 HTTP 进程分离。相同参数且尚未完成的请求共享 future；已完成的核验不会永久充当后续读取的凭证。读取进程损坏可由后续读请求重建，读请求不启动测试和模型调用。

实现位置　history.py / combined；history.py / IndexSynchronizer；read_service.py / ProofReaders

第四章　工作台与业务实例

4.1　运行上下文与分项导航



图 4.1　工作台验收流程与检查结果

工作台以仓库与运行为导航上下文。流程页显示当前阶段、耗时、检查项及执行器，具体动作有独立原件入口。阶段排成可横向滚动的一行，窄屏仍保留阶段顺序。查看某个阶段时，读者可以沿检查行进入明细，同时保留所选运行。

表 4.1　工作台功能视图

视图	主要对象	复核动作
方案与用例	需求出处、中文场景、执行协议	核对预期值及来源，区分有效用例、排除项和设计缺口。
缺陷详情	候选组、发生位置、优先级、归因	对照实测与预期，查看关联原件与复核依据。
证据溯源	运行、任务、尝试、原件	查看原始报告或浏览器记录，区分当前核验结果与当时报告。

前端展示数量取自后端结果和相应统计口径。run_id、选中的标签、发现项和原件都属于导航上下文；同名测试不能跨运行替换。工作台的目录速度、详情核验速度、原件加载速度需分别观察。

前端实现：web/src/Workbench.tsx、ConsoleFlow.tsx、UniversalPlanDetails.tsx、Catalogue.tsx、evidence.ts。

4.2　Flask 教程删除流程的证据链

需求 B14-R3_delete_post 来自本次设计的文档条目 D128，引用的是教程里删除路由提交事务并重定向到文章列表的示例。模型把这一约定转换成“浏览器接受确认对话框后删除文章”用例，编号 B14-C3_delete_journey，设计来源运行 int-5740126cbe21420e。



图 4.2　Flask 文章删除前后的页面状态

表 4.2　浏览器删除流程的执行记录

执行节点	实际回执	可核对的原件
用户操作	作者完成注册、登录和创建，随后在编辑页面执行删除	逐步截图、trace.zip
确认对话框	出现“Are you sure?”，执行器记录响应 accept	journey-result.json 中的 dialogs
重定向与列表断言	浏览器返回首页；指向 /1/update 的链接数预期 0，实际 0	步骤 expect_delete_redirect 与 expect_no_update_link
用例终态	19 个浏览器步骤，4.953 秒；该条用例通过	journey-result.json、report.xml、network.har

用例复验取用归档模型原件，单独准备环境并实际执行。截图证明的是本次界面状态；要对数据库物理删除或跨用户授权下结论，需增加对应的独立观察。本例不替代 Flask 全仓验收。

归档定位：docs/design/d041/r4-model-tutorial-calibration/business/B14-C3_delete_journey/。设计来源见 docs/design/d041/r4-model-calibration-provenance.json。

第五章　测试报告

5.1　实仓业务验收与结果归因

下表来自 D043 对三份历史归档的复核。Flask 和 sqlite-utils 使用 D041 r6 记录；Fastify 使用同版本追加复跑。业务用例实际执行，模型设计复用此前封存的响应。本文编制时复查了归档材料，未重新执行这三次业务验收。

表 5.1　实仓业务验收结果

仓库与运行	通过	失败	异常	整体门禁
Flask · int-364f0c4ce50b4ef8	68	3	14	无法判定
Fastify · int-9ebeca89d0194c8d	16	1	2	无法判定
sqlite-utils · int-fe2b255558a6465f	64	6	2	无法判定

合计 176 条业务用例，148 条通过、10 条失败、18 条异常。Flask 同轮另有 126 项冻结需求，其中 67 已验证、14 排除、45 未验证。需求与用例是一对多关系，且结果契约的全部测试计数还含原生回归等任务；三种分母不可互换。

5.1.1　两类不同的失败

Flask 历史用例 B11-case_login_form 报告标题为空，但同次 HTML 和网络归档中可找到 Log In - Flaskr。原断言结果保留，复核意见指出标题读取适配有误。仅凭红色检查行归责给目标项目，会误导修复。

另一次教程 CLI 初始化检查出现应用上下文异常。后续独立诊断绕开平台场景执行器，直接调用公开测试接口，仍得到退出码 1 且数据库未创建。原来的用例问题分类因此被追加更正为待复核。它说明特定调用可以复现失败；整个框架的影响范围仍需另证。

依据：D041 的 r4-Flask 逐项复核.md、r6-cli-review-correction.json；D043 的 historical-verification.json 和三个结果契约。

5.2　反例回归与读取性能

本次制档在当前工作区执行五个 pytest 模块：结果契约、发现归类、引文校验、Python 导出索引及历史索引。实测 146 项通过，耗时 4.68 秒，有一条 Starlette 测试客户端的依赖弃用提示。测试范围限于所列模块，不能据此判定任意仓库可验收通过。

表 5.2　反例回归设计

反例或故障设计	要求观察的行为
报告原件被替换	先从接口读取有效结果；更改临时测试归档的原件再读取。历史 allow 保留，当前变为 inconclusive，运行信封字节不变。
缺任务、重复身份、必验用例无结果	返回结构阻断；跳过或异常不算已验证。错误文本中含 assert 不能自动确认为缺陷。
历史索引重建或翻页	索引只投影历史元数据；查询条件与游标绑定；证据核验仍从原始归档读取。

5.2.1 规模基准的适用条件

历史基准在 Apple M5 Pro、48 GiB 内存、本地 SQLite WAL 环境中运行。数据为合成目录元数据，采用 TestClient 发起请求，每个规模取 30 次热查询，首页 50 条。下列是历史测量，独立于本次 pytest 回归。

表 5.2-2　历史目录规模基准

目录记录	冷建索引	热查询 P95	翻页唯一记录
1,000	3.949 秒	51.51 毫秒	1,000
10,000	23.297 秒	120.38 毫秒	10,000

10,000 条时该轮搜索耗时约 991 毫秒。目录测试没有运行模型和目标程序，也没有模拟多人远程访问。详情页还会读取原件，不能套用目录页的时延。更大证据集、跨机 I/O 和 24 小时持续负载仍需单独验证。

历史测量原件：docs/design/d043/history-benchmark.json。本次测试的命令、输出及源码摘要见随附技术图源与核查材料包。

第六章　部署与使用

6.1　单机运行环境

当前工作区使用 Python 3.12、Node.js 24 系列及 Docker。后端依赖见 requirements.lock.txt；工作台与验证工具分别使用 web 和 validation 目录的 npm 锁文件。以下为现有 macOS 工作区的操作路径，干净机器交付验证仍属后续工作。

```
uv venv --python 3.12 .venv
uv pip install --python .venv/bin/python -r requirements.lock.txt
npm ci --prefix web
npm run build --prefix web
```

按检查类型准备执行器镜像和浏览器运行环境。integrations/README.md 说明镜像构建与工具链锁；scripts/integrations.py provision 检查本机可用镜像并登记，不会自动构建全部镜像。模型设计另需有效连接；凭据通过受控配置注入，不放入验收材料。

```
.venv/bin/python scripts/local_service.py start
.venv/bin/python scripts/local_service.py status --worker
```

该启动器依赖 macOS launchd，start 会先确保 worker 已加载，再启动 API 并核查 /api/health。默认访问 127.0.0.1:18890。健康接口可用仅说明服务已响应，不能证明模型余额、全部镜像及目标业务依赖均已就绪。

6.2 主要接口与操作顺序

表 6.2 主要服务接口

操作	接口	回读内容
选择远端版本	POST /api/acceptance/remote/refs	分支、默认分支与提交
识别并确认范围	POST /api/acceptance/plan	计划身份、摘要、检查与排除项
启动与跟踪	POST /api/acceptance/runs	202 返回运行和作业 ID；用作业事件查看进展
核查结论与原件	GET /api/runs/{id}/result-contract	获取当前结果，再按原件 ID 访问 artifacts 接口

API 调用者可从 GET /api/result-contract/schema 读取契约。归档路径、执行器镜像与服务启动前提以部署节点为准。

第七章 项目总结

7.1 工程贡献

项目把交付中的三个环节接到同一版本上：独立标准决定要检验什么，成熟工具记录实际发生的行为，结果契约确定这些材料能支持何种结论。独立需求、尝试编号及原件引用贯穿计划和报告，因此事后复核可以具体到一条引用、一个浏览器步骤、一份内容摘要。

工程取舍体现在数据处理细节。预期值与实际观察分离，避免模型在运行后修改答案；精确去重保留重复命中，便于核对计数；当前判定与历史判定并存，使证据失效或复核更正能被看见。通用协议覆盖不了的动作归入缺口，不为单个被验收项目临时降低标准。

7.2 支持边界与后续验证

表 7.2 原型边界与验证计划

边界	当前状态	后续验收条件
通用仓库接入	可识别声明和技术栈；远程仓库中的符号链接、子模块与 LFS 等结构存在限制	对支持矩阵的每类结构给出干净环境实测与失败样例。
业务与多服务	已有独立环境、Redis/Celery 和真实浏览器案例；协议与服务类别有限	增加执行器时同时验证就绪、隔离、失败归因及资源回收。
证据安全与开源交付	本机归档可核验；截图与网络原件可能含敏感数据	验证外发脱敏、保留策略、安装、升级回滚和许可证材料。

边界	当前状态	后续验收条件
持续运行与部署	已有单机进程故障和合成目录基准	在持续负载及不同主机上记录恢复时间、资源和真实执行成功条件。

当前定位为可运行的工程 Alpha。发布范围应以可复现的真实项目、明确的执行前提和经核验的结果为依据。参赛演示可以展示工作台、真实业务步骤及证据回放；独立复验与全仓门禁分别标注。

附录　实现与资料索引

A　代码与工程图定位

表 A　工程图与源码位置

图号	主要源码目录或文件	核查内容
2.1、2.2	acceptance/api.py、jobs.py、read_service.py	接入分派、创建作业、读进程与执行进程职责
3.1、3.2	universal/service.py、discovery.py、generation.py	冻结计划、DAG、逐项验证及模型回执复用
3.3	universal/case_environment.py、orchestration.py	专属工作卷、私有网络、服务探针与清理
3.5	integrations/storage.py、store.py；results.py	内容寻址分片、原件元数据、观察与需求引用
3.7	results.py、integrations/gates.py、gates.rego	结构阻断、规则计算与三态门禁
3.8、3.9	jobs.py、history.py、read_service.py	尝试保护、恢复协调、索引与重新核验

简写文件名均相对于 acceptance/。随附包提供九幅黑白工程图的 SVG 源文件，可放大阅读或继续编辑；源码定位清单记录本次核查文件的摘要。

B　验证材料与关联文档

实仓执行与复核记录

结果契约、索引与恢复验证

产品需求文档与发布条件

GitLab 单测报告　/　报告与作业状态

Allure 测试状态　/　失败与异常

Playwright Trace Viewer　/　浏览器回放